VISION TRANSFORMERS

Kourosh GERAYELI

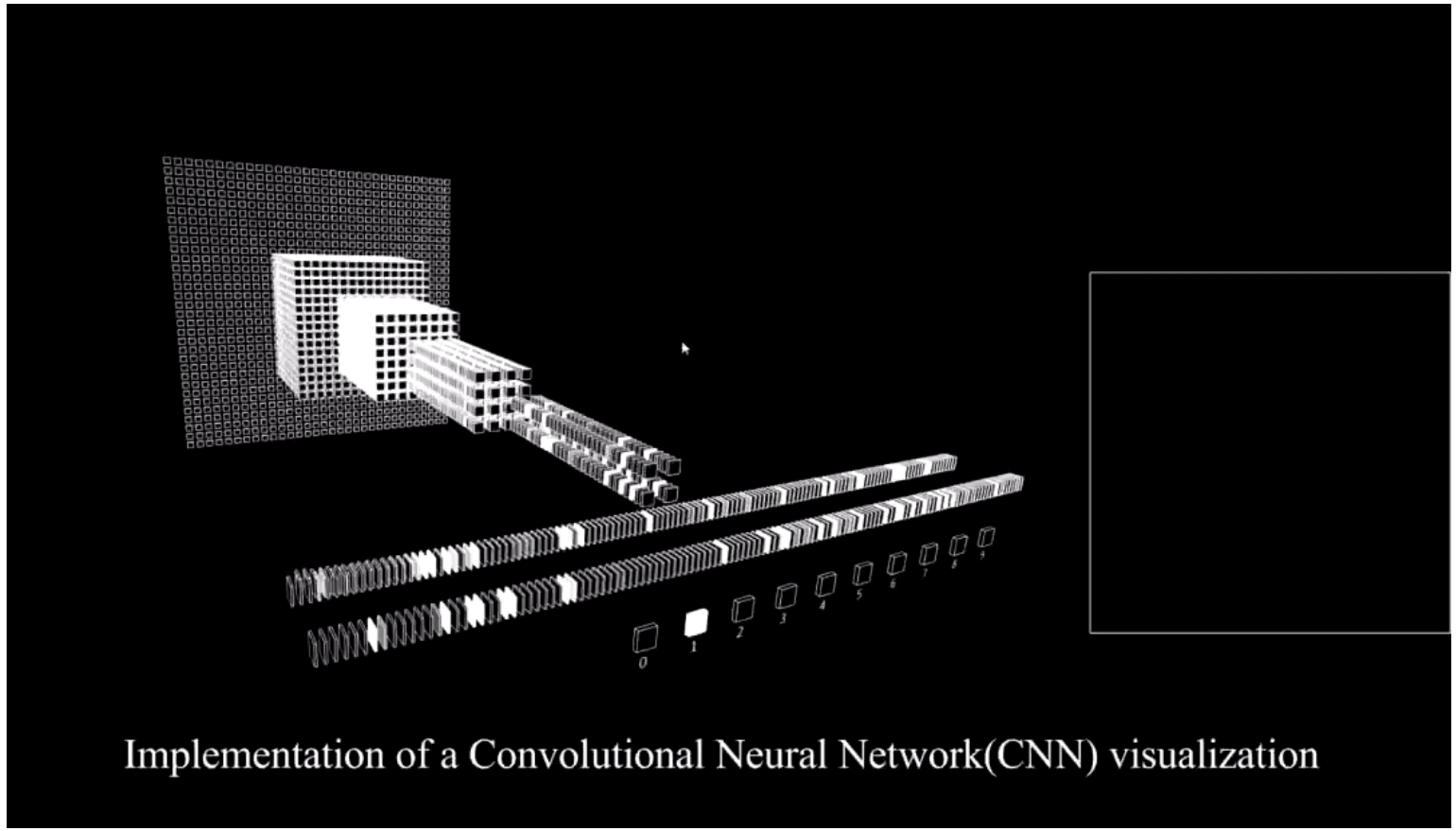
CEDRE 2026

TABLE OF CONTENTS

- Attention is all you need
- What is a transformer?
- Attention
- ViT
- CNN Vs ViT
- Notebook Yolo

What are the methods?

CNN



Are there any alternative methods?

Attention Is All You Need

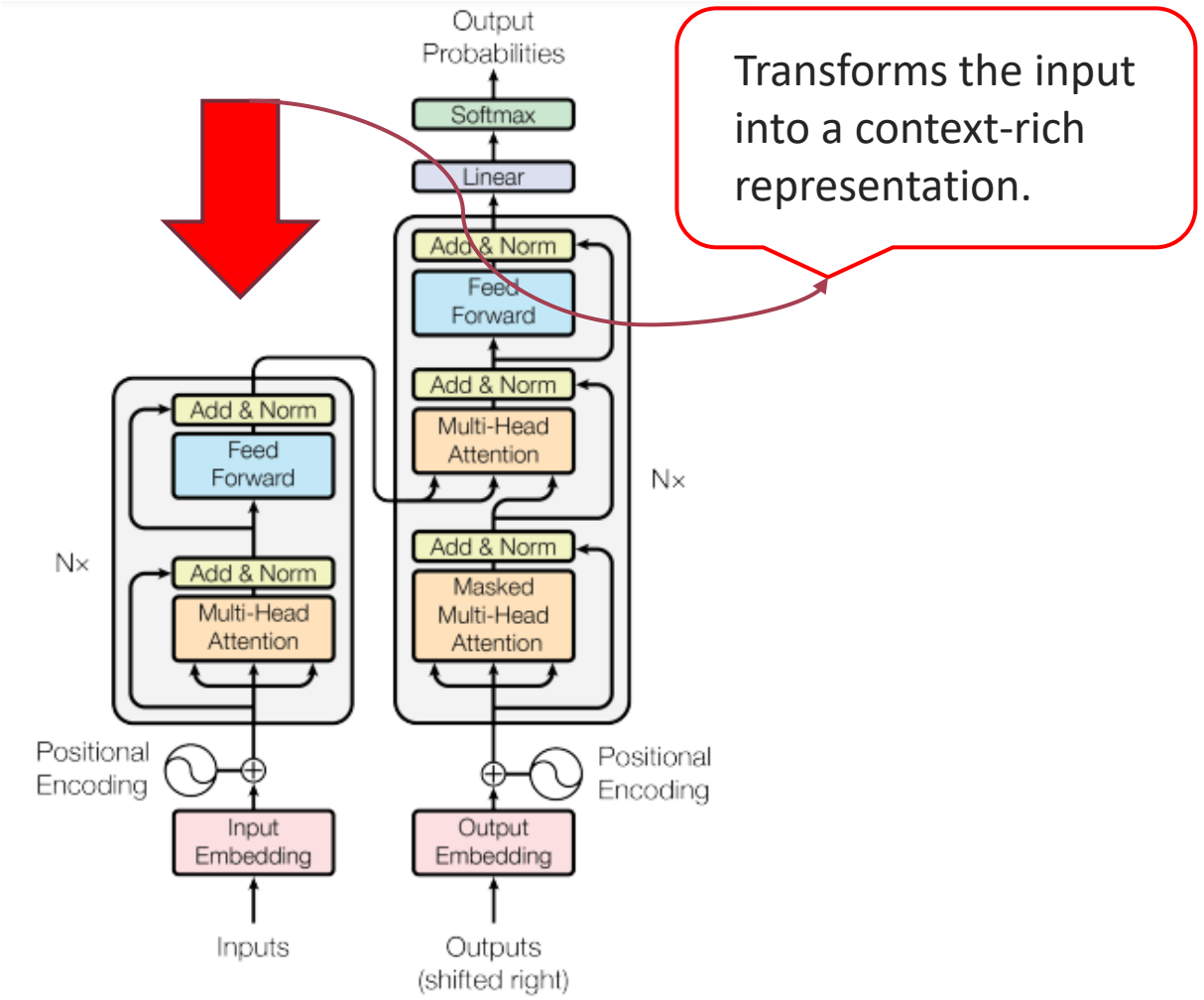
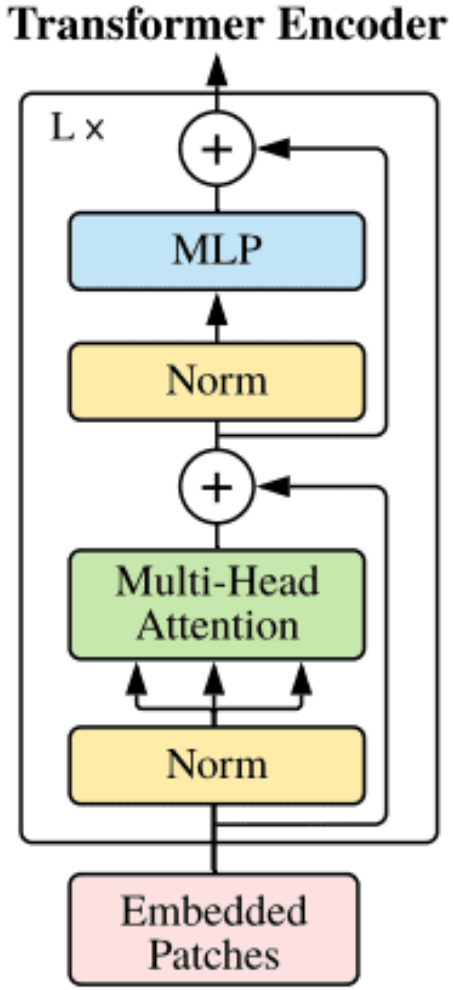


Figure 1: The Transformer - model architecture.

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale



"An Image is Worth 16*16 Words: Transformers for Image Recognition at Scale," published at ICLR 2021
Attention Is All You Need 2017
Both by google researchers

Transformers have proven to be pivotal in the development of sharper Natural Language Processing (NLP) models, while also extending its employability in additional areas within machine learning.

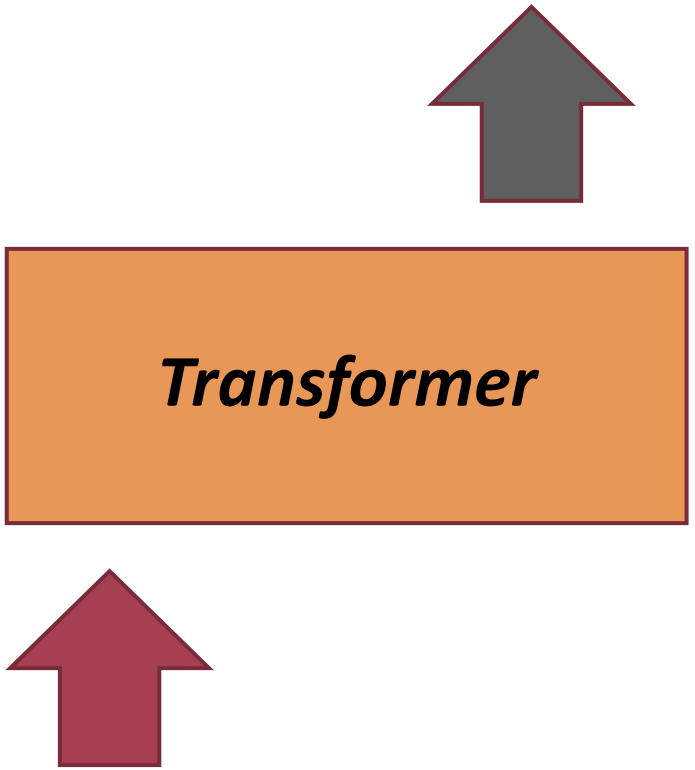
The transformer is an architecture that relies on the concept of **attention**, a technique used to provide weights to different parts of an input sequence so that a better understanding of its underlying context is achieved. This allows transformers to perform **machine translation**, **text generation** and many other NLP tasks.

As the Transformers architecture scaled well in Natural Language Processing, the same architecture was applied to images by creating small patches of the image and treating them as tokens.

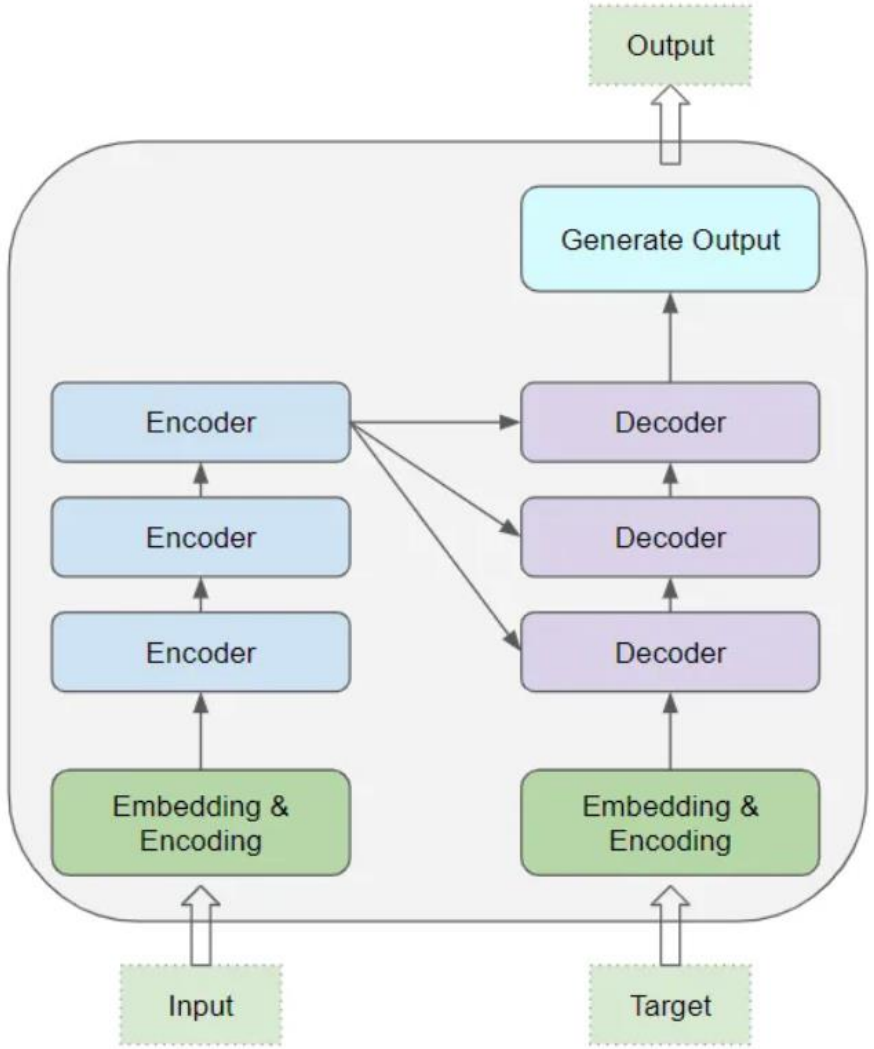
while transformers for NLP tasks consist of an encoder attention branch and a decoder attention branch, the ViT only uses an encoder. The output of the encoder is then passed to a neural network "head" that makes a prediction.

What is a transformer?

Output
« Comment tu t'appelles? »



Input
« What is your name? »



What is attention?

How?

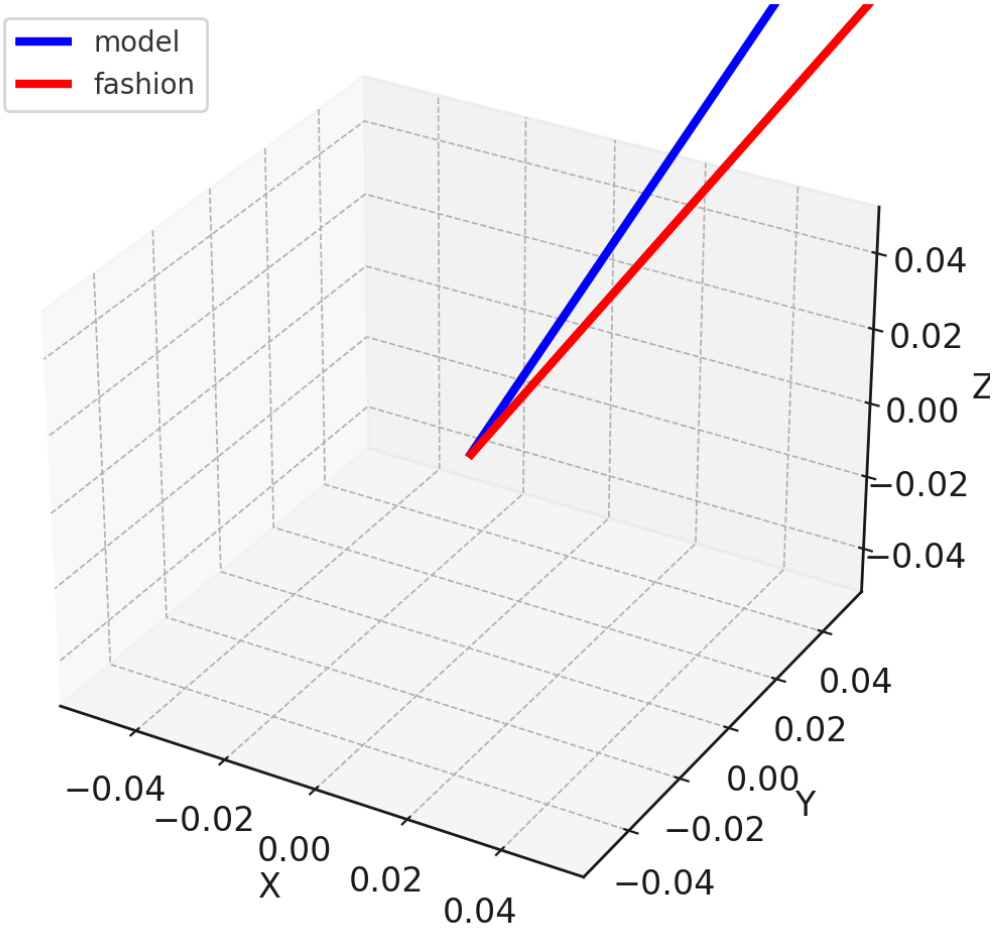
- **Tokenize**

The Fashion model fell on the scene!

- **Vector**

Fashion → $\begin{pmatrix} 1 \\ 5 \\ 15 \\ . \\ . \\ . \end{pmatrix}$

- **Dot product (Comparison)**



ViT (Visual Transformers)

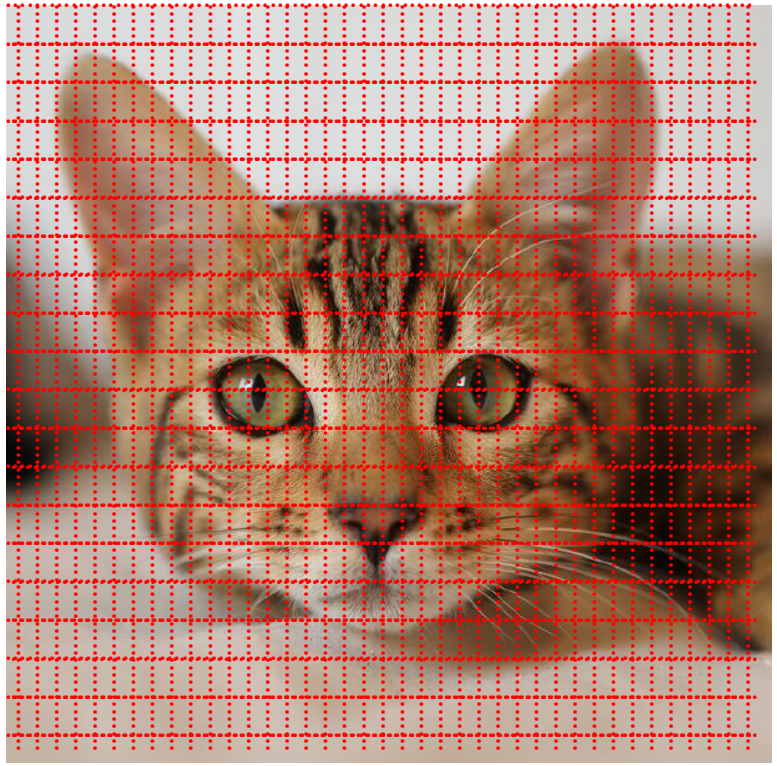
Vision Transformer (ViT) is a groundbreaking neural network architecture that reimagines how we process and understand images.

ViT introduces a new way to analyze images by dividing them into smaller patches and leveraging self-attention mechanisms.

This allows the model to capture both local and global relationships within images, leading to impressive performance in various computer vision tasks.

256

256

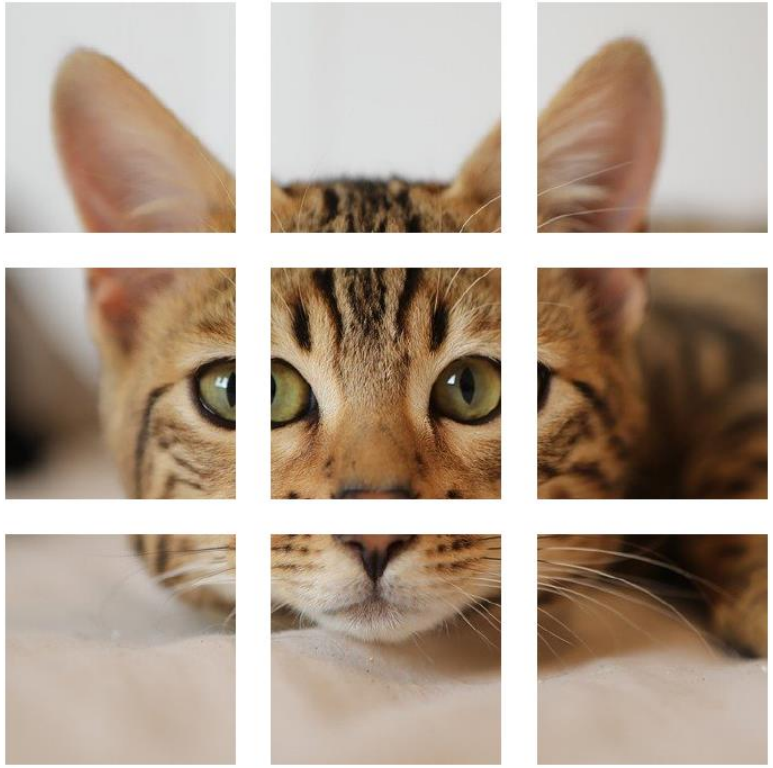


$$(256 * 256)^2 = 4{,}3\ B$$

Let's say that we have a low resolution image of size 256*256 which is not very big at least to human eye. We feed it to transformers and attention blocks

4,3 B comparison is a lot! So what should we do?

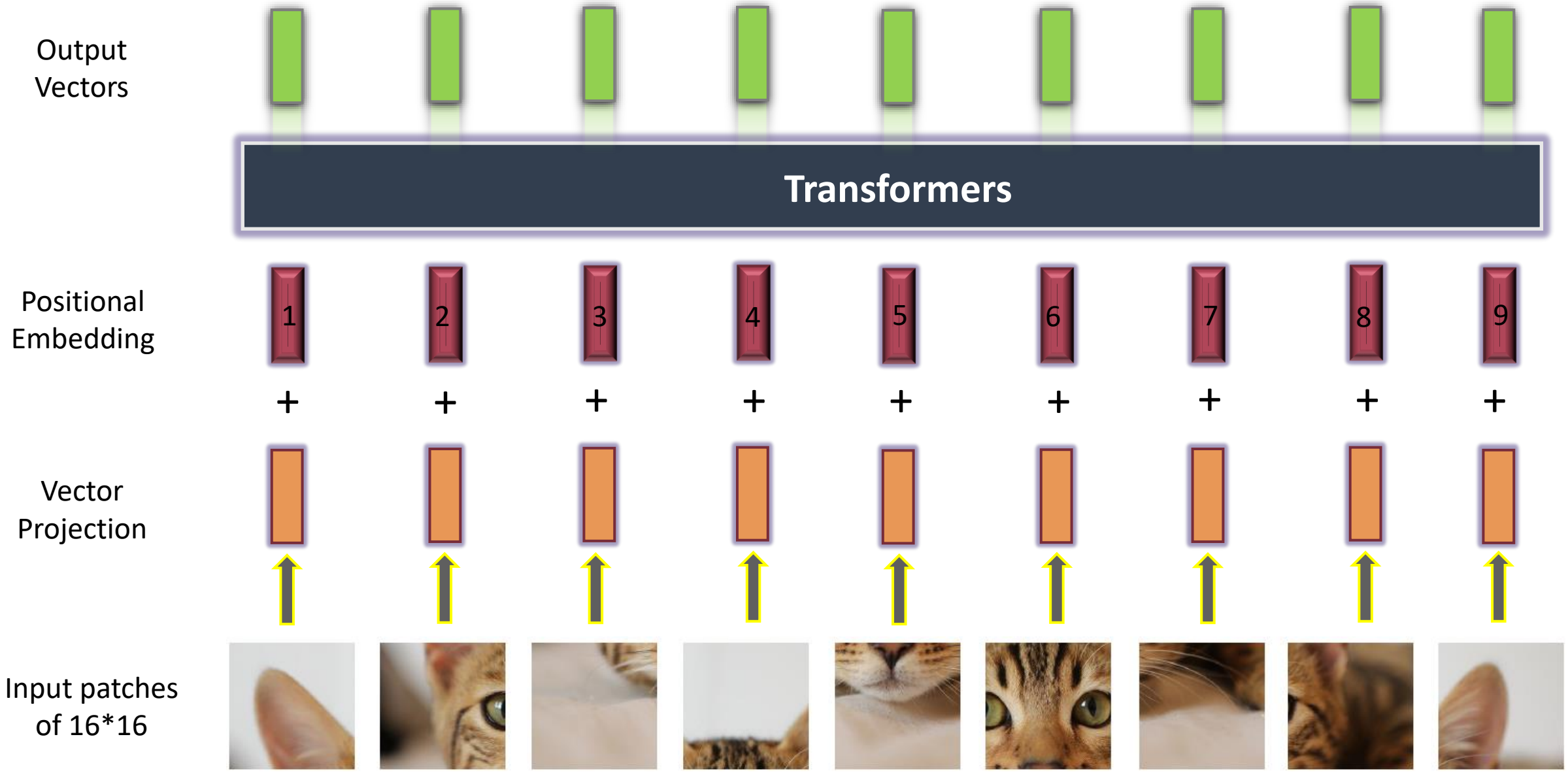
Every single pixel has to attend to every other single pixel



Vision Transformers reduce computation by **turning image patches into tokens**, so attention operates on **hundreds of tokens instead of tens of thousands of pixels**.

256 input patches : 16*16

65 500 Comparisons!!


Output Vectors
Transformers
Positional Embedding
1
2
3
4
5
6
7
8
9
+
Vector Projection
Input patches of 16*16

By the picture, we see that the input image (a) is “cut” into sub-images equally sized.
Each such sub-image goes through a linear embedding. From then on, each sub-image is just a one-dimensional vector.

A positional embedding is then added to these vectors (tokens). The positional embedding allows the network to know where each sub-image is positioned originally in the image.

Inside of a transformer :
These tokens are then passed, together with a special classification token, to the transformer encoders blocks, were each is composed of : A Layer Normalization (LN), followed by a Multi-head Self Attention (MSA) and a residual connection. Then a second LN, a Multi-Layer Perceptron (MLP), and again a residual connection. These blocks are connected back-to-back. Finally, a classification MLP block is used for the final classification only on the special classification token, which by the end of this process has global information about the picture.

Classification token :
Simply put, this is a special token that we add to our model that has the role of capturing information about the other tokens. This will happen with the MSA block (later on). When information about all other tokens will be present here, we will be able to classify the image using only this special token. The initial value of the special token (the one fed to the transformer encoder) is a parameter of the model that needs to be learned.
This is a cool concept of transformers! If we wanted to do another downstream task, we would just need to add another special token for the other downstream task (for example, classifying a digit as higher than 5 or lower) and a classifier that takes as input this new token. Clever, right?

When should we use ViT?

- **Large Datasets**: require large datasets to perform well, rely more heavily on massive amounts of data to learn visual features.
- **Global Context is Important**: Effective at capturing long-range dependencies across the entire image, so they perform well on tasks where understanding the full spatial context matters (e.g., scene understanding, complex object relationships).
- **Advanced Hardware**: More computationally expensive and benefit from running on powerful GPUs or TPUs.
- **Flexible Input Sizes**: Don't necessarily require resizing images to specific dimensions.
- **Higher-Resolution Images**: ViTs can often handle high-resolution images better than CNNs since they don't rely on downsampling as much, preserving more image details.
- **Transfer Learning from Large-Scale Pretraining**: Access to ViTs pretrained on large datasets like ImageNet-21k, they can generalize well on moderately sized datasets through transfer learning, leveraging the large pre-trained knowledge base.

CNN VS ViT

CNN	ViT
Process **raw pixel values** directly	Divides the input image into **patches** and transforms them into tokens
Uses **Convolutional** and **pooling** layers	Employs **attention mechanisms** to consider relationships among all patches
Relies on pooling layers for coarse **global information**.	Inherently captures global context through self-attention → recognizing relationships between **distant patches**
Requires **large amounts of labeled data** for training	Benefits from **pre-training datasets**

Appendix

Inductive bias

Inductive bias is a term used in machine learning to describe the set of assumptions that a learning algorithm uses to make predictions. In simpler terms, inductive bias is like a shortcut that helps a machine learning model make educated guesses based on the information it has seen so far.

Here's a couple of inductive biases we observe in CNNs:

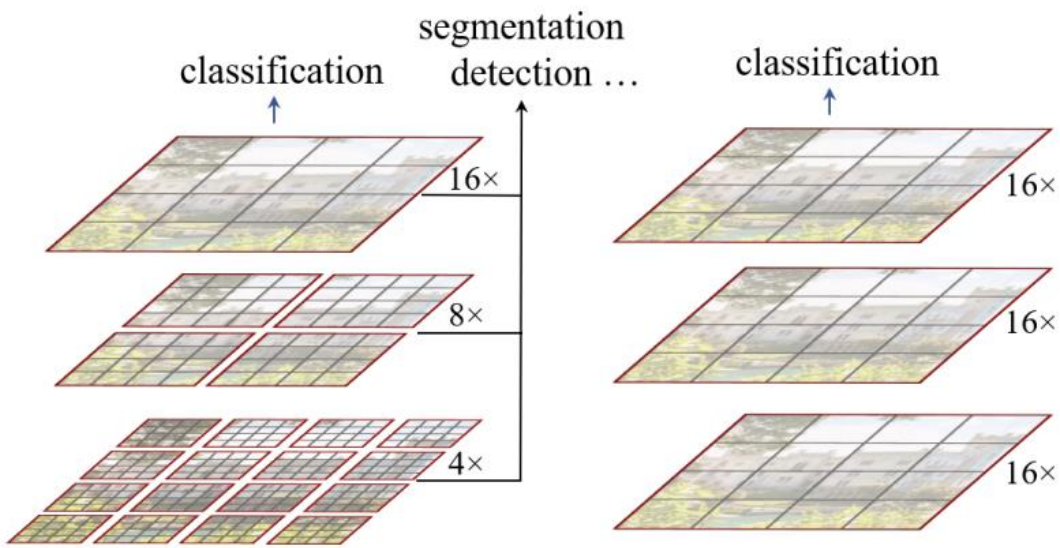
- Translational Equivariance: an object can appear anywhere in the image, and CNNs can detect its features.
- Locality: pixels in an image interact mainly with its surrounding pixels to form features.

CNN models are very good at these two biases. ViT do not have this assumption. That is why for a dataset size up to a certain threshold actually CNNs are better than ViT. But ViT has another power! The transformer architecture being (mostly) different types of linear functions allows ViT to become highly scalable. And that in turn allows ViT to overcome the problem of not having the above two inductive biases with massive ammount of data!

Swin Transformer

Swin Transformer is a hierarchical architecture whose representation is computed by **shifting window,** which brings greater efficiency by limiting self-attention computation to non-overlapping local windows while also allowing for cross-window connection.

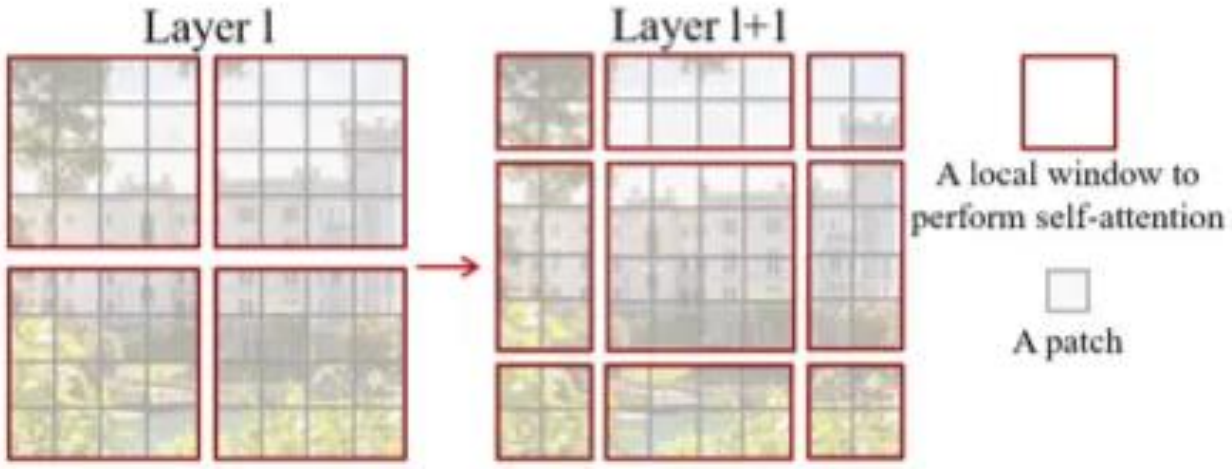
⇒ Swin Transformer is suitable as a general-purpose backbone for various vision tasks, in contrast to previous Transformer architecture (ViT) which produces feature maps of a single resolution and has quadratic complexity.



(a) Swin Transformer (ours) (b) ViT

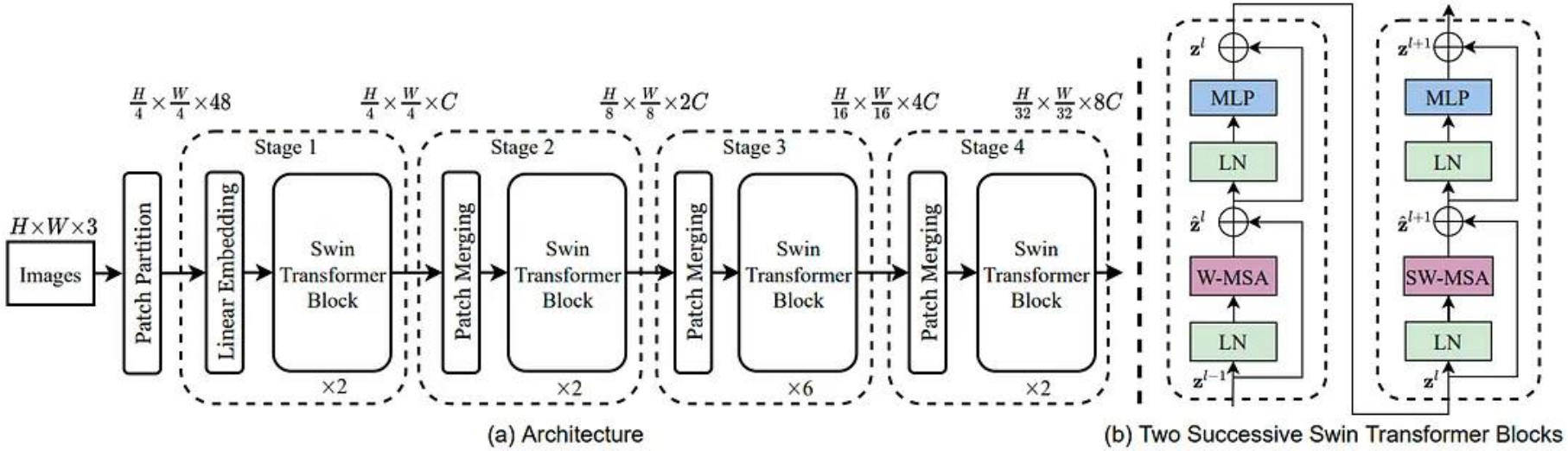
How does Swin Transformer work ?

A key design element of Swin Transformer is its shift of the window partition between consecutive self-attention layers.



As shown in Fig, in layer *l*, Swin Transformer splits input data into non-overlapping patches/windows using a patch splitting module, and self-attention is computed within each patch/window. In the next layer *l+1*, the window partitioning is shifted, resulting in new windows, self-attention is then calculated in each new window.

Swin Transformer Architecture



(a) Architecture (b) Two Successive Swin Transformer Blocks

First, an input image is passed through a “Patch Partition”, to split it into fixed-sized patches.
A patch size of 4×4 is used and the “Patch Partition” gives us *H/4 x W/4* patches. Each patch has a channel dimension of **4x4x3=48 pixels**.

2.1. Stage 1

To transform each patch from 48 pixels to a better size ***C***, a “Linear Embedding” layer is applied to **project it to an arbitrary dimension C.**
The new feature map is passed through a “Swin transformer block” (the size of inputs and outputs remains the same) → Output dimension: *H/4 x W/4 x C*

2.2. Stage 2

The feature map of size (*H/4 x W/4 x C*), is passed through a “Patch Merging” layer, which combines 2x2 neighboring windows and makes a new one, downsampling the resolution by 2x and increasing the feature map depth by 2 → Output size: (*H/8 x W/8 x 2C*)
This feature map is passed through another Swin transformer block, which keeps its dimension intact.

2.3. Stage 3 & 4

Stage 3 and 4 repeat the same procedure as stage 2, with output resolutions of *(H/16, W/16)* and *(H/32, W/32)*, respectively.
These stages jointly produce a hierarchical representation, with the same feature map resolutions (resolution is decreasing and its depth is increasing) as those of typical convolutional networks, such as VGGNet and ResNet, which can **conveniently replace the backbone networks in existing methods** for various vision tasks.

Notebook swin